PHENIKAA UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONIC ENGINEERING

COURSE PROJECT REPORT
REINFORCEMENT LEARNING

Title of the project:
Topic 13 – Multi-Delivery Robot:
Learning the Pickup and Delivery Order of a Multi-Parcel Robot with Tabular Q-Learning, SARSA and Monte Carlo ES

Student: Nguyen Quang Trung **Student ID:** 22011211
Supervisor: MSc. Vu Hoang Dieu **Major:** Electrical and Electronic Engineering

Score

Lecturer 1	Lecturer 2
MSc. Vu Hoang Dieu	

Hanoi, November 2026

Learning the Pickup and Delivery Order of a Multi-Parcel Robot with Tabular Q-Learning, SARSA and Monte Carlo ES

Nguyen Quang Trung
Student ID: 22011211
Phenikaa University
Hanoi, Vietnam

***Abstract*—A robot on a grid must pick up several parcels and deliver each one to its own drop cell, choosing the order by itself. We model the task as a deterministic Markov decision process with four maps of increasing difficulty: more parcels, a capacity of one, a delivery deadline, and a booster cell that doubles the capacity for one trip. A deadline clock and a booster flag are added to the state to keep it Markov, which brings the largest map to 357,696 encoded states (15,612 reachable). We compare three tabular methods, Q-learning, SARSA and Monte Carlo with exploring starts (MC ES), trained with exploring starts, algorithm-specific exploration schedules and a numba-compiled training loop that is about 150–180 times faster than a Python loop. Accuracy is measured as the share of start cells from which the greedy policy reaches the optimal return computed by value iteration. All three methods reach 100% accuracy on all four maps in 5/5 seeds. Q-learning learns fastest, MC ES is second and SARSA needs about three times as many episodes. Changing the late penalty, the deadline or the booster cell changes the learned delivery order exactly as the optimal solution predicts. Started in every reachable state rather than only at a start cell, Q-learning and MC ES are never wrong, while SARSA takes a slightly longer route from 0.3% of the states on level 3 and 1.8% on level 4, which we trace to a few dozen wrong decisions identified by their state codes.**

***Index Terms*—reinforcement learning, Q-learning, SARSA, Monte Carlo exploring starts, pickup and delivery, tabular methods**

I. Introduction

In a pickup and delivery problem, a vehicle collects goods at given locations and brings each item to its own destination [1]. The order of visits decides the route length, and constraints such as a vehicle capacity or a delivery deadline make some orders infeasible or costly. Instead of solving the routing problem directly, this work lets an agent learn the order from rewards alone.

The environment extends the Taxi domain [2], [3] from one passenger to several parcels, and adds a capacity limit, a deadline and a booster cell. Because the environment is small and deterministic, value iteration gives the exact optimal policy, which makes it possible to measure how close each learned policy is to the optimum, not only whether it reaches the goal.

The contributions are:

- a `gymnasium` environment [3] with four levels, from easy to very hard, each adding one mechanic, and a Markov state that includes a deadline clock and a booster flag;
- a comparison of Q-learning [4], SARSA [5] and MC ES [6] under the same exploring-starts protocol, with exploration schedules chosen per algorithm;
- a strict accuracy measure over every start cell, and the number of episodes each method needs to reach 100% on each level;
- experiments that change the reward and the environment, and value heatmaps that show how the deadline and the booster appear in the learned values;
- a check of every reachable state that finds where each policy goes wrong, level by level, and traces every mistake to a state code, together with a keyboard game and a desktop app that use the same rules.

II. Environment and MDP Formulation

A. Rules and Rewards

The robot moves on an $H \times W$ grid with walls. Each parcel i has a pickup cell and a drop cell. There are six actions: Up, Down, Left and Right move one cell (a move into a wall or off the grid leaves the robot in place), Pickup takes a waiting parcel in the current cell if the capacity allows it, and Drop delivers a carried parcel in its drop cell. Every step costs -1, a delivery earns $+20$, a wrong Pickup or Drop costs an extra -10, and a deadline parcel that is still undelivered when its deadline expires costs -30 once. An episode terminates when every parcel is delivered and is truncated after `max_steps` steps (100, 150, 200 and 300 on the four maps).

B. State and Markov Property

A state is the tuple

$$s = (r,\, c,\, \sigma_1, \ldots, \sigma_N,\, \tau,\, b), \tag{1}$$

where (r, c) is the robot cell, $\sigma_i \in \{0, 1, 2\}$ means that parcel i is waiting, carried or delivered, $\tau \in \{0, \ldots, T_d\}$ counts the steps toward the deadline T_d, and $b \in \{0, 1, 2\}$ means that the booster is unused, active or used up. Without τ, two visits to the same cell with the same parcel statuses could have different futures, and without b the result of Pickup would depend on the history; both are therefore part of the state. Once the deadline parcel is delivered, τ is set to T_d because time no longer

affects the reward, which merges states that would otherwise be duplicated.

C. Encoding and State-Space Size

The parcel statuses are read as a base-3 number, and the tuple is packed into one integer that indexes the Q table:

$$\text{code} = \textstyle\sum_{i=1}^{N} \sigma_i \, 3^{\,i-1}, \tag{2}$$

$$\text{idx} = \big(((rW + c)\, 3^N + \text{code})\, T + \tau\big) B + b, \tag{3}$$

with $T = T_d + 1$ on maps with a deadline ($T = 1$ otherwise) and $B = 3$ on the map with a booster ($B = 1$ otherwise). The encoded size is $|S| = H\,W\,3^N\,T\,B$. Far fewer states are reachable (Table I): walls cannot be occupied, a capacity of one excludes two carried parcels, and τ and b only take values that can actually occur. The reachable set is found by breadth-first search.

D. Four Levels

The four maps S1–S4 are four levels of increasing difficulty: level 1 (easy), level 2 (medium), level 3 (hard) and level 4 (very hard). Each level adds one mechanic to the previous one and the grid grows from 5×5 to 8×8 (Fig. 1, Table I). Every map is a JSON file, so a level can be changed without changing the code. The robot always starts at S in the top-left cell $(0, 0)$; cells are written as (row, column) with row 0 at the top, pickup cells in lowercase and drop cells in uppercase. The optimal route of each level, computed by value iteration, is:

- *S1, level 1 (easy),* 5×5*, capacity 2.* Parcel A goes from a $(0, 4)$ to A $(4, 0)$ and B from b $(4, 4)$ to B $(0, 2)$; a short wall in column 2 blocks the direct path. The robot picks up a (step 5) and b (step 10), carries both, and delivers A (step 15) and then B (step 22): 22 steps, return 18. The new mechanic is only that two parcels can be carried together.
- *S2, level 2 (medium),* 6×6*, capacity 1.* Three parcels, A: $(0, 4) \to (5, 0)$, B: $(3, 0) \to (1, 4)$, C: $(5, 5) \to (2, 2)$, and more walls. Since only one parcel fits, the order matters: the robot delivers B (picked up at step 4, delivered at step 11), then A (13, 23), then C (29, 36): 36 steps, return 24.
- *S3, level 3 (hard),* 7×7*, capacity 1.* A: $(0, 6) \to (2, 0)$, B: $(2, 3) \to (0, 3)$, and C: $(6, 6) \to (6, 0)$ with a 20-step deadline and a -30 penalty. The robot goes straight to the bottom-right corner and delivers C at step 20, exactly on the deadline, then B (28, 31) and A (35, 44): 44 steps, return 16. Without the deadline, $B \to A \to C$ would take only 40 steps.
- *S4, level 4 (very hard),* 8×8*, capacity 1 (2 with the booster).* A: $(0, 7) \to (7, 0)$, B: $(1, 7) \to (7, 1)$, C: $(4, 2) \to (0, 3)$, D: $(7, 7) \to (4, 4)$ with a 22-step deadline, and the $\times 2$ booster cell at $(3, 0)$. Entering the booster cell lets the robot carry two parcels; the trip ends the first time it is empty-handed after a Drop, and the cell works once per episode. The robot first delivers D (picked up at step 15, delivered at step 22, exactly on the deadline), then C (27, 33), switches the booster on at step 39, picks up A (50) and B (52) in the top-right corner and delivers B (65) and A (67) in the bottom-left corner: 67 steps, return 13. Without the booster cell the best route needs four separate trips and 75 steps.

Levels 3 and 4 therefore contain trade-offs that a greedy nearest-parcel rule would miss: the deadline parcel must come first although it is far away, and the detour to the booster cell pays off by 8 steps.

III. Learning Algorithms

All three methods keep a table $Q(s, a)$ of size $|S| \times 6$ and act greedily with respect to it after training, $\pi(s) = \arg\max_a Q(s, a)$. They differ only in the target used to update Q.

A. Exploring Starts and Exploration Schedules

Every training episode starts in a reachable state drawn uniformly at random (any parcel statuses, τ and b), and its first action is also random. Every pair (s, a) is therefore updated directly, including states such as "carrying C at $\tau = 15$" that a robot starting at $\tau = 0$ rarely visits. Evaluation still starts with every parcel waiting. After the first action, Q-learning and SARSA act ε-greedily with an exponentially decaying ε,

$$\varepsilon(ep) = \varepsilon_s \left(\frac{\varepsilon_e}{\varepsilon_s}\right)^{\min(1,\, ep/(f\, n_{ep}))}, \tag{4}$$

where n_{ep} is the number of episodes and f the fraction of training over which ε decays (Table II).

B. Q-Learning

Q-learning [4] updates after every step toward

$$y = \begin{cases} r, & \text{if } s' \text{ is terminal,} \\ r + \gamma \max_{a'} Q(s', a'), & \text{otherwise,} \end{cases} \tag{5}$$

with $Q(s, a) \leftarrow Q(s, a) + \alpha\,[y - Q(s, a)]$. A truncated episode is not terminal, so the bootstrap term is kept on its last step. Because the target uses the best next action, Q-learning learns Q^* while it explores (off-policy). The environment is deterministic, so the target has no noise and $\alpha = 1$ is used.

C. SARSA

SARSA [5] replaces the maximum by the action a' that the ε-greedy policy actually takes in s':

$$y = r + \gamma\, Q(s', a'). \tag{6}$$

It learns the value of the policy it is running (on-policy) and converges to Q^* only when exploration vanishes in the limit (GLIE) [7]. The target is noisy through a', so α decays linearly from 0.5 to 0.1, and ε decays to 10^{-5} during the first half of training.

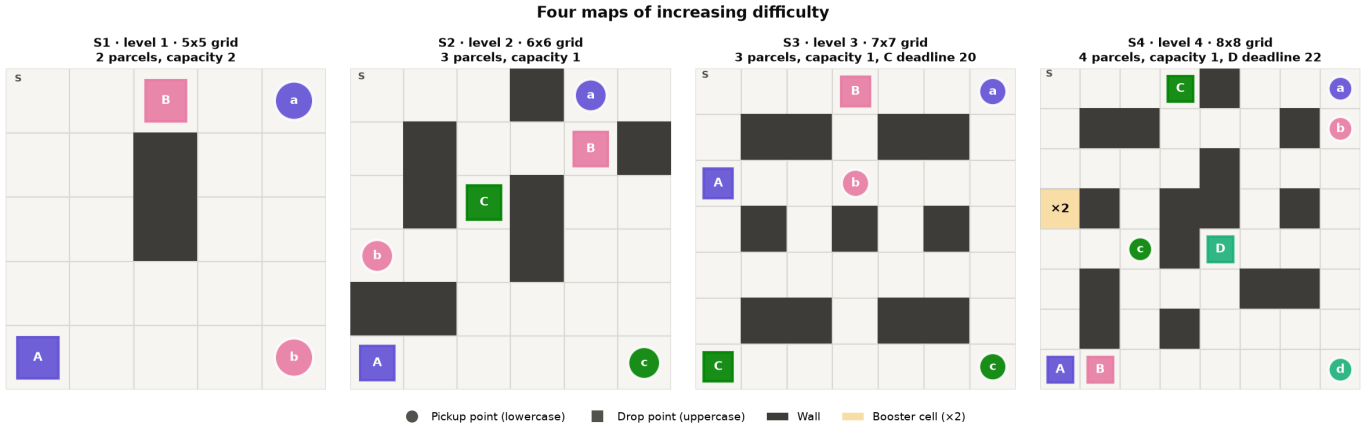


Fig. 1. The four maps by level, from level 1 (easy, S1) to level 4 (very hard, S4). The robot starts at S. Pickup cells are circles (lowercase letters), drop cells are squares (uppercase letters), walls are dark and the $\times 2$ booster cell is yellow.

TABLE I
THE FOUR LEVELS, THEIR STATE-SPACE SIZE AND OPTIMAL ROUTES (VALUE ITERATION)

Map	Level	Grid	Parcels	Capacity	New mechanic	Encoded $\lvert S\rvert$	Reachable	Optimal route (steps, return)	max_steps
S1	1 (easy)	5×5	2	2	two parcels carried together	225	186	$A \to B$ (22, 18)	100
S2	2 (medium)	6×6	3	1	one parcel at a time, order matters	972	535	$B \to A \to C$ (36, 24)	150
S3	3 (hard)	7×7	3	1	parcel C within 20 steps	27,783	4,027	$C \to B \to A$ (44, 16)	200
S4	4 (very hard)	8×8	4	1 (2 with booster)	booster cell and parcel D within 22 steps	357,696	15,612	$D \to C \to B \to A$ (67, 13)	300

D. Monte Carlo with Exploring Starts

MC ES [6] runs the greedy policy ($\varepsilon = 0$) after the random first pair, computes the returns backwards,

$$G_t = r_{t+1} + \gamma\, G_{t+1}, \tag{7}$$

and updates the first visit of each pair, $Q(s,a) \leftarrow Q(s,a) + \alpha\,[G_t - Q(s,a)]$. All exploration comes from the random start. With a greedy policy in a deterministic environment, the return after (s,a) is a fixed number for the current policy, so $\alpha = 1$ is used instead of the sample average $1/N$, which would keep the very negative returns of early, looping policies for a long time. A looping policy is cut at `max_steps` with a very negative return, so the action that caused the loop is penalised on the next visit.

E. Optimal Reference and Accuracy

Value iteration [8] over all reachable states,

$$V(s) \leftarrow \max_a \big[r(s,a) + \gamma\, V(s')\big], \tag{8}$$

gives the optimal return and Q^*. It is used only as a reference. The *accuracy* of a policy is the share of start cells c (every empty cell, every parcel waiting) from which the greedy policy obtains exactly the optimal return $R^*(c)$:

$$\mathrm{Acc} = \frac{1}{|C|} \sum_{c \in C} \mathbf{1}\big[R^{\pi}(c) = R^*(c)\big]. \tag{9}$$

This is stricter than checking the single start cell S: a policy can follow one memorised route from S and still fail from other cells.

F. Implementation

Since the environment is deterministic, the transition function is precomputed once into three arrays: next state $NS[s,a]$, reward $R[s,a]$ and termination flag $D[s,a]$. The training loops only look up these arrays and are compiled with numba [9]. On S2 with 20,000 episodes, one Q-learning run drops from 10.7 s to 0.06 s and one MC run from 15.4 s to 0.10 s, a speed-up of about 150–180 times. Table II lists the final settings, which were tuned on the same four maps with 5 seeds.

G. Playing the Levels and the Desktop App

Two tools use exactly the same `env.step()` as the agents. In `play.py` a person plays any level with the keyboard (W/A/S/D to move, J to pick up, K to drop) on a text map, and at the end the score is compared with the optimal route; this makes the rules and the trade-offs of levels 3 and 4 easy to check by hand. `app.py` is a Tkinter desktop app: it trains any algorithm on any level with the same compiled loops while the map and the learning curves update live, overlays the greedy policy, the value $V = \max_a Q$ or the cells where the greedy action is not optimal under Q^*, and lets the user drive the robot, step the agent, or jump to any reachable state by typing its state code idx from (3), showing Q and Q^* for the six actions.

IV. EXPERIMENTS AND RESULTS

A. Setup

Each algorithm is trained on each map with 5 seeds. Every run is evaluated 200 times; each evaluation runs the greedy policy from cell S (return and success) and from every start cell (accuracy).

TABLE II
HYPERPARAMETERS

	Q-learning	SARSA	MC ES
γ	0.99		
α	1	$0.5 \to 0.1$ (linear)	1
ε	$1 \to 0.01$	$1 \to 10^{-5}$	0
ε decay, f	exponential, 0.8	exponential, 0.5	–
Start state	exploring starts (random state and first action)		
Q_0	0		
Episodes	S1: 100k, S2: 300k, S3: 2M, S4: 6M		
Seeds	5 per algorithm and map		
Evaluation	200 times per run (greedy, $\varepsilon = 0$)		

TABLE III
EPISODE AT WHICH ACCURACY FIRST REACHES 100% (RANGE OVER 5 SEEDS)

Map (opt.)	Q-learning	SARSA	MC ES
S1 (18)	0.5k	5.5k–33.5k	7.0k–9.5k
S2 (24)	1.5k–3.0k	64.5k–159k	30k–51k
S3 (16)	340k–400k	980k–1.22M	260k–370k
S4 (13)	840k–1.44M	3.45M–4.14M	1.47M–2.04M

B. Accuracy

All 60 runs (3 algorithms $\times$ 4 maps $\times$ 5 seeds) end with 100% accuracy and the optimal return from S, delivering on time on S3 and S4 (Fig. 2). The learned Q tables also converge to Q^*: the final mean error $|Q - Q^*|$ is 0 for Q-learning and MC ES and between 0.001 and 0.047 for SARSA. Table III shows when each method first reaches 100%. Q-learning is the fastest on three maps; MC ES is slightly faster on S3 (median 290k against 350k episodes) because its greedy training episodes are short. On S4 the medians are 1.14M episodes for Q-learning, 1.74M for MC ES and 3.57M for SARSA. One seed on S4 (6M episodes) takes 38 s with Q-learning, 23 s with SARSA and 38 s with MC ES, and the full set of experiments takes 221 s on a 12-core CPU.

C. Why the Settings Work

Exploring starts are the key setting. The τ and b dimensions multiply the number of states, and states away from the usual route, such as carrying C with the deadline clock at 15, would almost never be updated if every episode started at $\tau = 0$. Starting from a random reachable state with a random first action updates every pair (s, a) directly. The exploration schedule is chosen per algorithm: Q-learning learns Q^* whatever ε is, so ε only needs to reach 0.01; SARSA learns the value of its own ε-greedy policy, so ε must fall to 10^{-5} for it to match Q^*; MC ES explores only through exploring starts. The step size follows the noise of the target: $\alpha = 1$ when the target is noise-free (Q-learning, greedy MC ES) and a decaying α for SARSA.

D. Comparison of the Three Algorithms

Since all three methods reach the optimum, they are compared on speed, stability and sensitivity to settings (Table IV). *Q-learning* is the fastest and the most stable: it learns Q^* from every step, including random exploratory steps, and its target is noise-free, so $\alpha = 1$ gives the exact value at once. *MC ES* is second. It updates only at the end of an episode, which slows the early phase, but greedy returns in a deterministic environment are exact, so it converges to Q^* like Q-learning. It depends entirely on exploring starts, its only source of exploration. *SARSA* is the slowest, needing about three times as many episodes as Q-learning on S4. It learns the value of its exploring policy and matches Q^* only once ε is very small, and its noisy target makes actions with nearly equal values swap, so its accuracy still dips after first reaching 100%. Per second, however, a SARSA episode is the cheapest. Its usual advantage, safer behaviour while exploring, does not appear here because the maps contain no dangerous cells.

E. Learned Policy and Value Function

On S4 the learned policy matches the optimal solution (Fig. 3): the robot picks up D in the bottom-right corner and delivers it in the middle of the map before its deadline, delivers C on the top row, goes down to the $\times 2$ cell to switch the booster on, picks up A and B in the top-right corner, and delivers both in the bottom-left corner, finishing in 67 steps with return 13. The agent has learned both trade-offs: it prioritises the deadline parcel although it is far away, and it accepts a detour to use the booster.

Fig. 4 colours every cell by the learned value $V(s) = \max_a Q(s, a)$ for a start with every parcel waiting. The value at S is 13.7, 17.8, 8.6 and 2.5 on S1–S4, below the undiscounted optimal returns (18, 24, 16, 13) because later rewards are discounted by $\gamma = 0.99$. Fig. 5 shows the value in four phases of the S4 route. Within a phase, cells near the current target are darker, so the greedy robot walks toward darker cells. The deadline appears as a cliff: in phase 2 (carrying D, $\tau = 15$) the cells far from D's drop cell fall to about -11, because from there D cannot arrive in time and the -30 penalty is certain. In phase 8 the robot carries A and B together, and almost the whole map is dark because two $+20$ rewards are a few steps away.

F. Changing the Reward and the Environment

Three variants were trained with Q-learning and 5 seeds (Table V, Fig. 6). When the late penalty on S3 is removed, being late costs nothing, so the agent drops its priority for C and takes the 4-step-shorter order $B \to A \to C$. When the deadline is relaxed to 45 steps, delivering C last is still on time, so the agent also switches to $B \to A \to C$; the reachable states grow from 4,027 to 14,558, and all seeds remain optimal. Without the booster cell, S4 needs four separate trips ($D \to B \to C \to A$, 75 steps). In every case the learned policy matches the optimal solution in 5/5 seeds.

G. Where Does the Robot Go Wrong?

Accuracy only starts the robot with every parcel waiting, $\tau = 0$ and the booster unused. A policy can be right there and wrong in states it seldom reaches, for example after it has been pushed off its route. We therefore start the greedy policy in every reachable unfinished state (184 on S1, 532 on S2, 4,024 on S3 and 15,604 on S4) and call a state wrong when

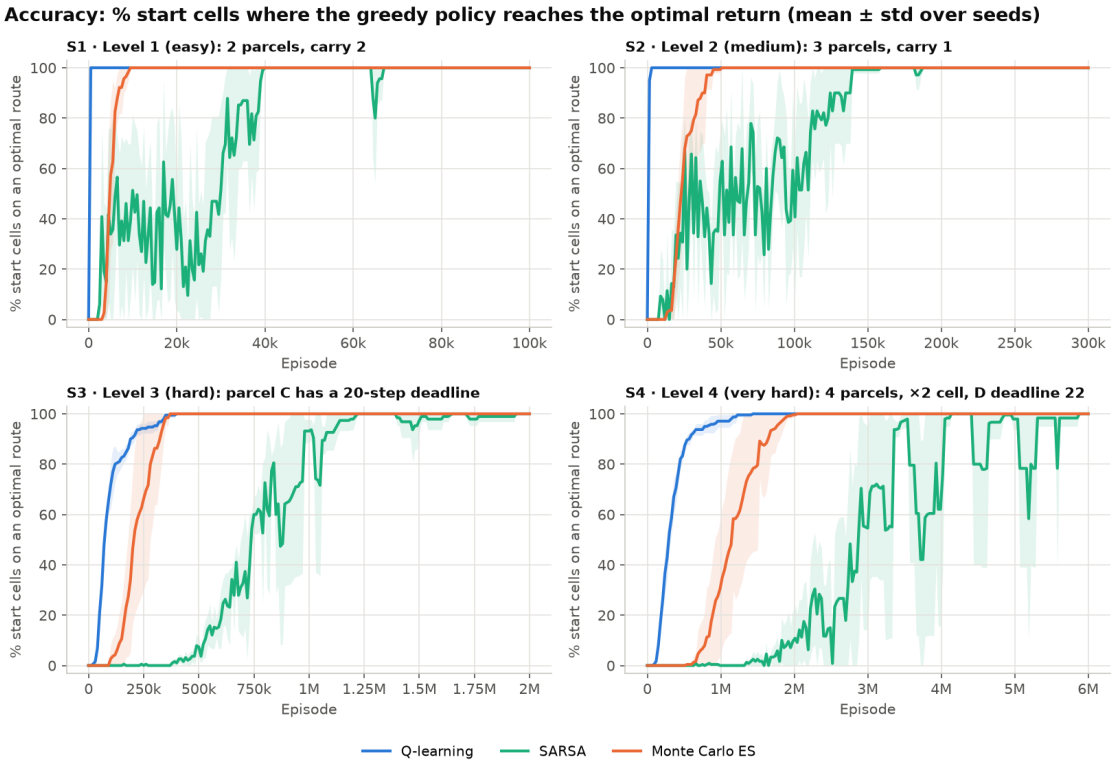


Fig. 2. Accuracy over training episodes: share of start cells from which the greedy policy reaches the optimal return, mean ± standard deviation over 5 seeds.

TABLE IV
COMPARISON OF THE THREE ALGORITHMS (MEDIANS OVER 5 SEEDS WHERE APPLICABLE)

Criterion	Q-learning	SARSA	MC ES
Update	after every step	after every step	after the episode
Target	$r + \gamma \max Q(s')$, no noise	$r + \gamma Q(s', a')$, noisy through a'	return G, no noise with $\varepsilon = 0$
Learns the value of	the optimal policy (off-policy)	its ε-greedy policy (on-policy)	its greedy policy (on-policy)
Episodes to 100% on S4	1.14M	3.57M	1.74M
Mean accuracy during training (S1/S2/S3/S4)	0.99 / 0.99 / 0.95 / 0.94	0.77 / 0.77 / 0.60 / 0.48	0.95 / 0.91 / 0.89 / 0.80
Drops below 100% after reaching it	never	9–19% of evaluations	never
Final mean error $\lvert Q - Q^* \rvert$	0	0.001–0.047	0
Time per seed on S4	38 s	23 s	38 s
Wrong if started anywhere (S3/S4)	0 / 0	0.3 / 1.8%	0 / 0

TABLE V
REWARD AND ENVIRONMENT VARIANTS (Q-LEARNING, 5 SEEDS)

Variant	Optimum (VI)	Learned
S3 base: deadline 20, late penalty 30	$C \to B \to A$, 44 steps, return 16	optimal, on time
S3: late penalty 0	$B \to A \to C$, 40 steps, return 20 (late)	optimal
S3: deadline 45	$B \to A \to C$, 40 steps, return 20 (on time)	optimal
S4 base: booster cell	$D \to C \to B \to A$, 67 steps, return 13	optimal
S4: no booster cell	$D \to B \to C \to A$, 75 steps, return 5	optimal

the policy collects less than the optimal return from it. Each wrong state is classified as never finishing, a late delivery, an illegal Pickup/Drop or a longer route, and the greedy route is followed to the first state whose action is not optimal under Q^*, the *wrong decision*, which is reported by its state code. Checking one Q table takes less than a second.

TABLE VI
WRONG STATES WHEN THE ROBOT IS STARTED IN EVERY REACHABLE STATE (RANGE OVER 5 SEEDS; MEAN SHARE)

Level (states)	Q-learning	SARSA	MC ES
1, S1 (184)	0	0	0
2, S2 (532)	0	0	0
3, S3 (4,024)	0	0–46 (0.3%)	0
4, S4 (15,604)	0	273–301 (1.8%)	0

Table VI gives the results per level. Q-learning and MC ES are right in every reachable state on all four levels in 5/5 seeds. SARSA is right everywhere on levels 1 and 2, but takes a longer route from 0.3% of the states on level 3 and 1.8% on level 4, losing at most 6 points and never looping or delivering late.

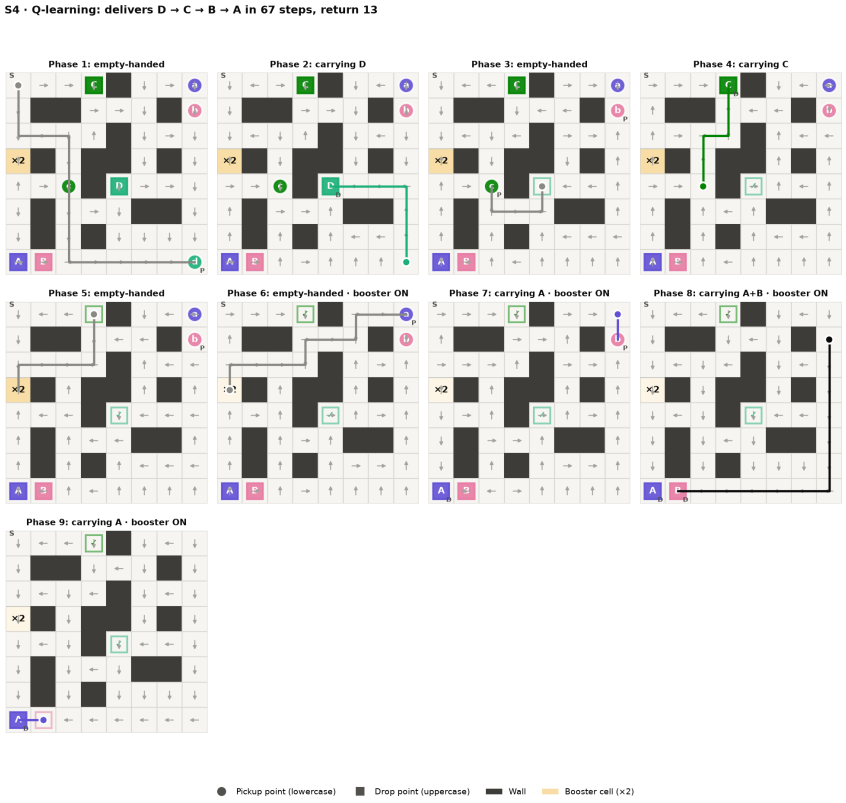


Fig. 3. S4 policy learned by Q-learning, one panel per phase (parcel and booster status). Grey arrows: greedy action in each cell; P/D: Pickup/Drop. The coloured line is the actual route, coloured by the parcel carried (black: two parcels).

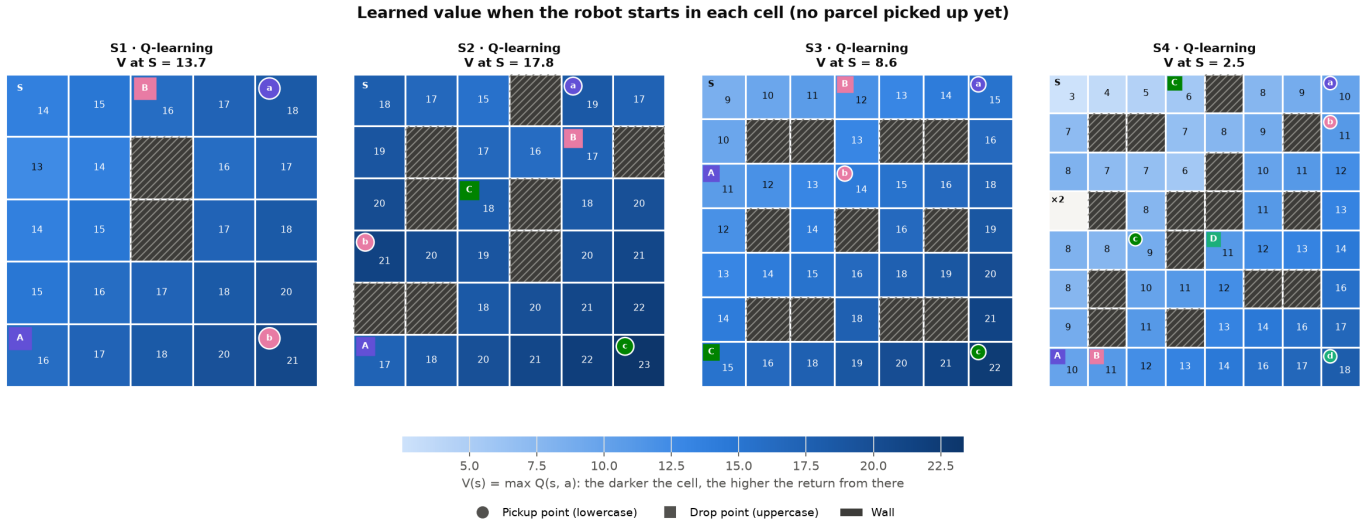


Fig. 4. Learned start value $V(s) = \max_a Q(s, a)$ in every cell of the four maps (Q-learning, no parcel picked up yet). Darker cells promise a higher return; hatched cells are walls.

Fig. 7 shows the costliest SARSA state on level 4: the robot is at $(0, 0)$ carrying C with the booster on and $\tau = 7$. SARSA delivers C at once, which ends the booster trip, and needs 61 steps (return -11); the optimal route first fetches A while still holding C and then carries A and B together (55 steps, return -5). The wrong decision is at state code 17419, $(0, 3, 0, 0, 1, 0, 10, 1)$ on C's drop cell, where SARSA drops C instead of moving down; the two actions differ by only 0.04 in Q^*, less than the error of SARSA's Q values. All 301 wrong states of this seed come from 39 wrong decisions on two cells, C's pickup and drop cells. Twice as many episodes do not help (1.8%), whereas a final step size of 0.02 halves the

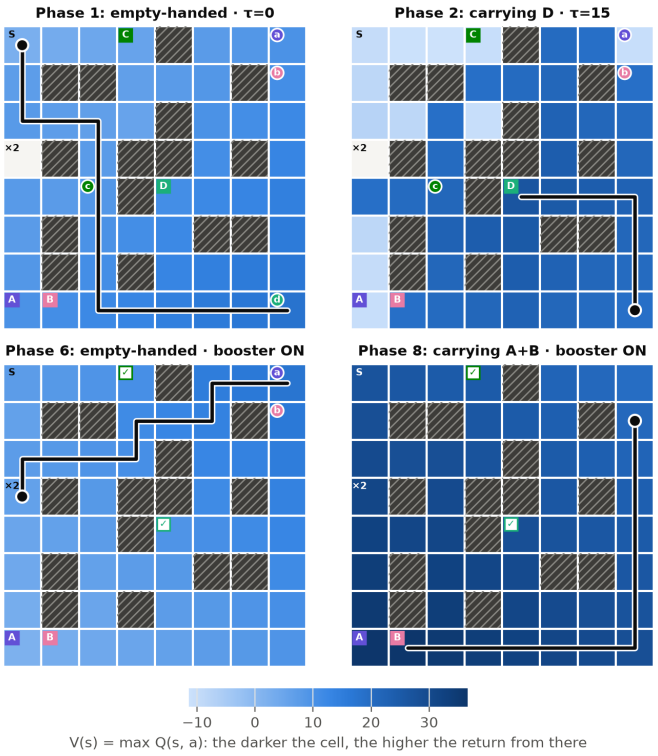


Fig. 5. Learned value in four of the nine phases of the S4 route; the black line is the robot's route in that phase. In phase 2 the cells that can no longer meet D's deadline have negative values.

wrong states on level 4 and removes them on level 3, which points to the noise of SARSA's target rather than to too little training.

V. Discussion

The results hold for a deterministic environment. There, Q-learning and MC ES benefit from noise-free targets; with stochastic moves (for example wheel slip) this advantage would shrink and the ranking could change. Exploring starts require a simulator that can place the robot in any state, which is easy here but not on a real robot. The deadline clock makes the state space grow linearly with the deadline length, and a Q table is tied to one grid, so every map is trained separately. Finally, the hyperparameters were tuned on the same four maps that are reported.

VI. Conclusion

Tabular Q-learning, SARSA and MC ES all learn the optimal pickup and delivery order on four maps of increasing difficulty, reaching 100% accuracy over every start cell in 5/5 seeds. Exploring starts and algorithm-specific exploration schedules are the key settings that let every state be learned despite the deadline clock and the booster flag. Q-learning learns fastest and most stably, MC ES is second, and SARSA needs about three times as many episodes. The learned policies react to changes in the late penalty, the deadline and the booster cell exactly as the optimal solution predicts, and the learned values make the deadline visible as a region of negative value. Checking every reachable state shows that Q-learning and MC ES are optimal everywhere, while SARSA's few remaining mistakes on levels 3 and 4 come from a few dozen near-ties in Q^*. Future work includes a stochastic version of the environment, Expected SARSA and Q(λ), and function approximation so that one model can serve several maps.

References

[1] S. N. Parragh, K. F. Doerner, and R. F. Hartl, "A survey on pickup and delivery problems. Part II: Transportation between pickup and delivery locations," *J. für Betriebswirtschaft*, vol. 58, no. 2, pp. 81–117, 2008.

[2] T. G. Dietterich, "Hierarchical reinforcement learning with the MAXQ value function decomposition," *J. Artif. Intell. Res.*, vol. 13, pp. 227–303, 2000.

[3] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[4] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[5] G. A. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," Cambridge Univ. Eng. Dept., Tech. Rep. CUED/F-INFENG/TR 166, 1994.

[6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[7] S. Singh, T. Jaakkola, M. L. Littman, and C. Szepesvári, "Convergence results for single-step on-policy reinforcement-learning algorithms," *Machine Learning*, vol. 38, no. 3, pp. 287–308, 2000.

[8] R. Bellman, *Dynamic Programming*. Princeton, NJ, USA: Princeton Univ. Press, 1957.

[9] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proc. 2nd Workshop LLVM Compiler Infrastructure in HPC*, 2015, pp. 1–6.

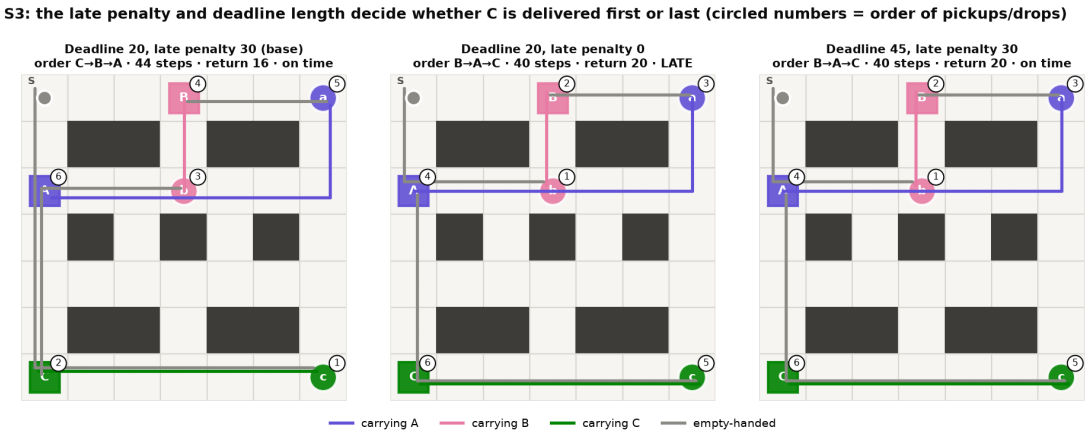


Fig. 6. S3 with three configurations. The base map delivers C first; with no late penalty or a 45-step deadline the order becomes $B \rightarrow A \rightarrow C$. Circled numbers give the order of pickup and drop events.

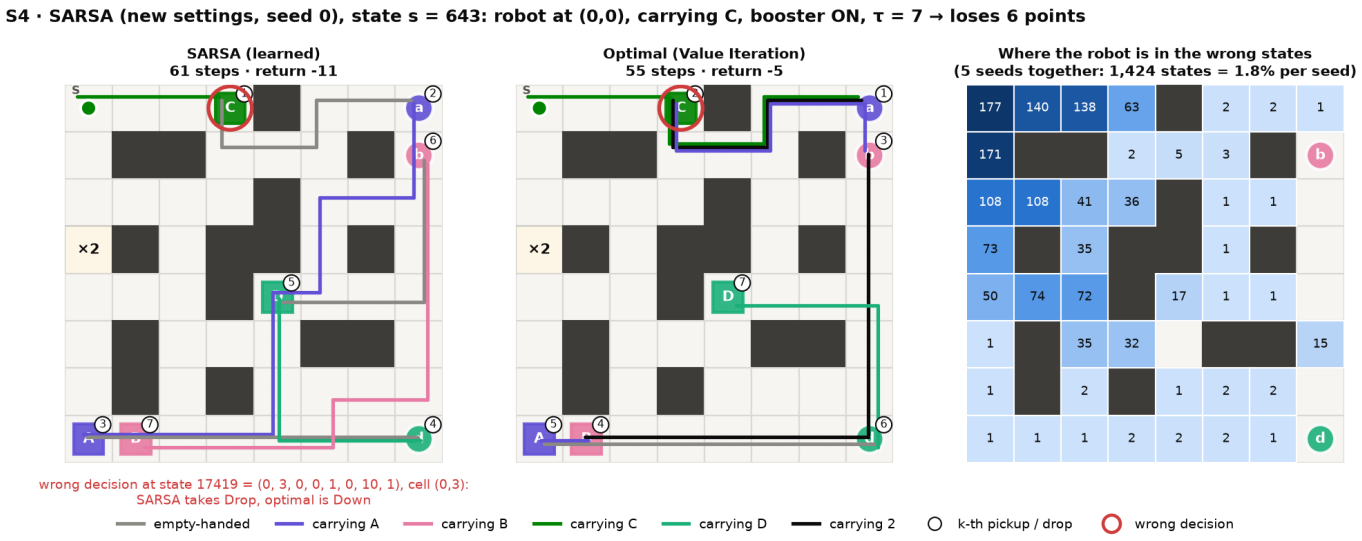


Fig. 7. Level 4, SARSA (seed 0): the costliest wrong state, learned route (left) against the optimal route (middle), with the wrong decision circled in red; right: robot position in the wrong states of all 5 seeds.